安全中台大模型

项目报告

项目名称：4

报告生成时间：2024-12-18 13:07:20

第一部分：漏洞扫描结果汇总

1.项目基本情况

项目名称	4
检测文件名称	4
有漏洞文件数量	20
开发语言	java
扫描类型	
检测开始时间	2024-12-18 12:41:14
检测耗时	0时24分22秒

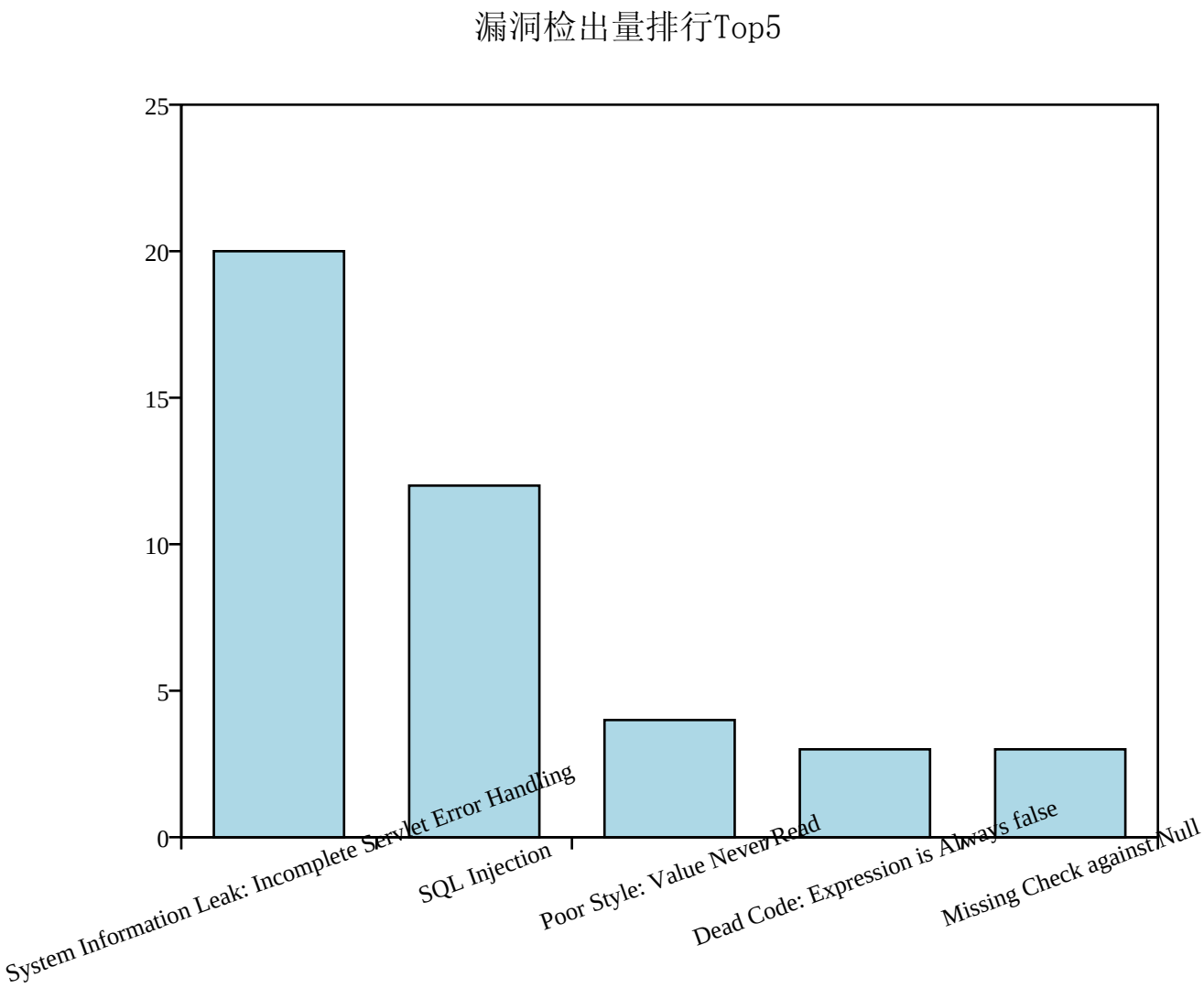
2.漏洞检测结果汇总

2.1 检测结果汇总表

漏洞检测结果详情统计

序号	漏洞名称	统计
1	Cookie Security: HTTPOnly not Set	1
2	Cookie Security: Persistent Cookie	1
3	Dead Code: Expression is Always false	3
4	Dead Code: Expression is Always true	2
5	Header Manipulation: Cookies	2
6	Missing Check against Null	3
7	Poor Style: Value Never Read	4
8	Resource Injection	1
9	SQL Injection	12
10	System Information Leak: Incomplete Servlet Error Handling	20

2.2 漏洞类型统计图


漏洞检出量排行Top5
25
20
15
10
5
0
System Information Leak: Incomplete Servlet Error Handling
SQL Injection
Poor Style: Value Never Read
Dead Code: Expression is Always false
Missing Check against Null

第二部分：漏洞扫描结果详情

1 越界读取

1.1 漏洞概述

漏洞编号	漏洞名称	风险等级
Cookie Security: HTTPOnly not Set	越界读取	中危

1.2 漏洞详情

1）漏洞定位

BenchmarkTest00111.java:37

漏洞分析

漏洞爆发点:userCookie = new Cookie(...)

爆发点函数:doGet()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServlet; import javax.servlet.http.HttpServletRequest; import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-00/BenchmarkTest00111") public class BenchmarkTest00111 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie userCookie = new javax.servlet.http.Cookie("BenchmarkTest00111", "bar");

```
userCookie.setMaxAge(60 * 3); // Store cookie for 3
minutesuserCookie.setSecure(true); userCookie.setPath(request.getRequestURI()
);userCookie.setDomain(new java.net.URL(request.getRequestURL().toString()).
getHost()); response.addCookie(userCookie);
javax.servlet.RequestDispatcherrd = request.getRequestDispatcher("/sqli-00/B
enchmarkTest00111.html");rd.include(request, response); } @Override public
voiddoPost(HttpServletRequest request, HttpServletResponse response)
throwsServletException, IOException { response.setContentType("text/html;cha
rset=UTF-8");
```

1.3 漏洞描述

软件将数据写入预期缓冲区的末尾或开始之前。通常，这可能导致数据损坏、崩溃或代码执行。该软件可以修改索引或执行指针算术，该指针算法引用缓冲区边界之外的内存位置。然后，后续的写入操作将产生未定义或意外的结果。

1.4 修复建议

1.使用不允许发生此弱点的语言，或提供使此弱点更容易避免的构造。例如，许多执行自己的内存管理的语言（如 Java 和 Perl）都不会受到缓冲区溢出的影响。其他语言（如 Ada 和 C#）通常提供溢出保护，但程序员可以禁用该保护。请注意，即使语言本身在理论上是安全的，语言与本机代码的接口仍可能被溢出。 2.使用不允许发生此弱点的经过审查的库或框架，或者提供使此弱点更容易避免的构造。例子包括Messier和Viega [REF-57]的安全C字符串库（SafeStr），以及Microsoft的Strsafe.h库。这些库提供容易溢出的字符串处理函数的更安全版本。 3.使用自动提供保护机制的功能或扩展来运行或编译软件，以缓解或消除缓冲区溢出。例如，某些编译器和扩展提供内置于已编译代码中的自动缓冲区溢出检测机制。

2 在网页生成过程中对输入的处理不当（跨站脚本攻击）

2.1 漏洞概述

漏洞编号	漏洞名称	风险等级
Cookie Security: Persistent Cookie	在网页生成过程中对输入的处理不当（跨站脚本攻击）	高危

2.2 漏洞详情

1）漏洞定位

BenchmarkTest00111.java:38

漏洞分析

漏洞爆发点:setMaxAge()

爆发点函数:doGet()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletRequest; import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-00/BenchmarkTest00111") public class BenchmarkTest00111 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie userCookie = new javax.servlet.http.Cookie("BenchmarkTest00111", "bar"); userCookie.setMaxAge(60 * 3); // Store cookie for 3 minutes userCookie.setSecure(true); userCookie.setPath(request.getRequestURI()); userCookie.setDomain(new java.net.URL(request.getRequestURL().toString()).getHost()); response.addCookie(userCookie); javax.servlet.RequestDispatcher

rd = request.getRequestDispatcher("/sqli-00/BenchmarkTest00111.html");rd.include(request, response); } @Override public voiddoPost(HttpServletRequest request, HttpServletResponse response) throwsServletException, IOException { response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie[] theCookies = request.getCookies();

2.3 漏洞描述

XSS全称为Cross Site Scripting，为了和CSS分开简写为XSS，中文名为跨站脚本。该漏洞发生在用户端，是指恶意攻击者往Web站点里插入恶意脚本代码，当用户浏览该网站时，嵌入其里面的脚本代码会被执行，从而达到恶意用户的特殊目的。网站里插入恶意脚本代码后，当用户浏览该网站时，嵌入其里面的脚本代码会被执行，从而达到恶意用户的特殊目的。根据是否写入数据看来分类主要有反射性和存储型两大类。 • 反射型跨站脚本（Reflected XSS） • 储存型跨站脚本（Persistent XSS） 区别：存储型XSS的恶意代码存在数据库里，反射型XSS的恶意代码存在URL里。

2.4 修复建议

1.在服务端进行输入验证（输入验证） 服务器后端进行输入验证，包括输入的数据类型、数据长度、数据格式、特殊字符等进行验证。 2.数据输出实体编码（输出编码） 输出编码手段主要有3种编码：URL编码、HTML编码和JavaScript编码。 JAVA：开源的ESAPI library；框架（Spring：HtmlUtils.htmlEscape） PHP：使用htmlspecialchars等函数进行过滤输出。

3 越界读取

3.1 漏洞概述

漏洞编号	漏洞名称	风险等级
ead Code: Expression is Always false	越界读取	中危

3.2 漏洞详情

1）漏洞定位

BenchmarkTest01381.java:80

漏洞分析

漏洞爆发点:IfStatement

爆发点函数:doSomething()

缺陷源:

漏洞源代码

} else throw new ServletException(e); } } // end doPost private class Test { public String doSomething(HttpServletRequest request, String param) throws ServletException, IOException { String bar; // Simple ? condition that assigns param to bar on false condition int num = 106; bar = (7 * 42) - num > 200 ? "This should never happen" : param; return bar; } } // end innerclass Test } // end DataflowThruInnerClass

2）漏洞定位

BenchmarkTest00840.java:75

漏洞分析

漏洞爆发点:IfStatement

爆发点函数:doPost()

缺陷源:

漏洞源代码

// parameter in query string. // And then check to see if its in the middle of the query string and if so, trim off what // comes after. int ampersandLoc = queryString.indexOf("&", paramLoc); if (ampersandLoc != -1)

{ param = queryString.substring(paramLoc + paramval.length(), ampersandLoc); } param = java.net.URLDecoder.decode(param, "UTF-8"); String bar; // Simple ? condition that assigns param to bar on false condition int num = 106; bar = (7 * 42) - num > 200 ? "This should never happen" : param; try { String sql = "SELECT * from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; org.owasp.benchmark.helpers.DatabaseHelper.JDBCtemp late.batchUpdate(sql); response.getWriter() .println("No results can be displayed for query: " + org.owasp.esapi.ESAPI.encoder().encodeForHTML(sql) + "<br>" + " because the Spring batchUpdate method doesn't return results."); // System.out.println("no results for query: " + sql + " because the Spring batchUpdate // method doesn't return results."); } catch (org.springframework.dao.DataAccessException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) {

3）漏洞定位

BenchmarkTest00203.java:56

漏洞分析

漏洞爆发点:IfStatement

爆发点函数:doPost()

缺陷源:

漏洞源代码

String param = ""; if (request.getHeader("BenchmarkTest00203") != null) { param = request.getHeader("BenchmarkTest00203"); } // URL Decode the header value since req.getHeader() doesn't. Unlike req.getParameter(). param = java.net.URLDecoder.decode(param, "UTF-8"); String bar; // Simple ?

condition that assigns param to bar on false condition int num = 106; bar = (7 * 42) - num > 200 ? "This should never happen" : param; String sql = " INSERT INTO users (username, password) VALUES ('foo','" + bar + "')"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.ge tSqlStatement(); int count = statement.executeUpdate(sql, new String[] {"USERNAME", "PASSWORD"}); org.owasp.benchmark.helpers.DatabaseHelper.output UpdateComplete(sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing request."); return; } else throw new ServletException(e); } }

3.3 漏洞描述

软件读取数据超过预期缓冲区的末尾或开始之前。 通常，这可能允许攻击者从其他内存位置读取敏感信息或导致崩溃。当代码读取可变数量的数据并假定存在一个哨兵来停止读取操作（如字符串中的 NUL）时，可能会发生崩溃。预期的哨兵可能不在越界内存中，从而导致读取过多的数据，从而导致分段错误或缓冲区溢出。该软件可以修改索引或执行指针算术，该指针算法引用缓冲区边界之外的内存位置。然后，后续读取操作将产生未定义或意外的结果。

3.4 修复建议

1.输入验证：假设所有输入都是恶意的。使用“接受已知良好”的输入验证策略，即使用严格符合规范的可接受输入的白名单。拒绝任何不完全符合规范的输入，或将其转换为符合规范的内容。执行输入验证时，请考虑所有可能相关的属性，包括长度、输入类型、可接受值的完整范围、缺失或额外的输入、语法、相关字段之间的一致性以及是否符合业务规则。作为业务规则逻辑的一个示例，“boat”在语法上可能是有效的，因为它只包含字母数字字符，但如果输入只期望包含“红色”或“蓝色”等颜色，则它无效。不要完全依赖查找恶意或格式错误的输入（即，不要依赖黑名单）。黑名单可

能会错过至少一个不需要的输入，尤其是在代码的环境发生变化时。这可以为攻击者提供足够的空间来绕过预期的验证。但是，黑名单可用于检测潜在攻击或确定哪些输入的格式非常错误，应将其彻底拒绝。若要降低引入越界读取的可能性，请确保验证并确保对任何长度参数、缓冲区大小计算或偏移进行正确的计算。特别要小心，不要在不受信任的输入中依赖哨兵（即特殊字符，如 NUL）。 2.语言选择：使用提供适当内存抽象的语言。

4 输入验证不当

4.1 漏洞概述

漏洞编号	漏洞名称	风险等级
Dead Code: Expression is Always true	输入验证不当	高危

4.2 漏洞详情

1）漏洞定位

BenchmarkTest01560.java:74

漏洞分析

漏洞爆发点:IfStatement

爆发点函数:doSomething()

缺陷源:

漏洞源代码

return; } else throw new ServletException(e); } } // end doPost private class Test { public String doSomething(HttpServletRequest request, String param) throws ServletException, IOException { String bar; // Simple if statement that assigns param to bar on true condition int num = 196; if

((500 / 42) + num > 200) bar = param; else bar = "This should never happen"; return bar; } } // end innerclass Test } // end DataflowThruInnerClass

2）漏洞定位

BenchmarkTest00678.java:52

漏洞分析

漏洞爆发点:IfStatement

爆发点函数:doPost()

缺陷源:

漏洞源代码

@Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); org.owasp.benchmark.helpers.SeparateClassRequest scr = new org.owasp.benchmark.helpers.SeparateClassRequest(request); String param = scr.getTheParameter("BenchmarkTest00678"); if (param == null) param = ""; String bar; // Simple if statement that assigns param to bar on true condition int num = 196; if ((500 / 42) + num > 200) bar = param; else bar = "This should never happen"; String sql = "SELECT userid from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; try { // Long results = // org.owasp.benchmark.helpers.DatabaseHelper.JDBCtemplate.queryForLong(sql); Long results = org.owasp.benchmark.helpers.DatabaseHelper.JDBCtemplate.queryForObject(sql, Long.class); response.getWriter().println("Your results are: " + String.valueOf(results)); } catch

```
(org.springframework.dao.EmptyResultDataAccessException e) {
response.getWriter() .println( "No results returned for query: " +
org.owasp.esapi.ESAPI.encoder().encodeForHTML(sql));
```

4.3 漏洞描述

产品不会验证或错误地验证可能影响程序的控制流或数据流的输入。当软件未正确验证输入时，攻击者能够以应用程序其余部分未预料到的形式手工创建输入。这将导致系统的某些部分收到意外的输入，这可能导致控制流更改、对资源的任意控制或任意代码执行。

4.4 修复建议

1.使用输入验证框架，如 Struts 或 OWASP ESAPI Validation API。 2.了解不受信任的输入可以进入软件的所有潜在区域：参数或参数、Cookie、从网络读取的任何内容、环境变量、反向 DNS 查找、查询结果、请求标头、URL 组件、电子邮件、文件、文件名、数据库以及向应用程序提供数据的任何外部系统。 3.当应用程序合并来自多个源的数据时，请在合并源后执行验证。单个数据元素可能通过验证步骤，但在组合后违反了预期的限制。 4.在调用跨语言边界的代码（例如从解释型语言到本机代码）时，请特别小心验证所有输入。这可能会在语言边界之间产生意外的交互。确保您没有违反与之接口的语言的任何期望。例如，即使 Java 可能不容易受到缓冲区溢出的影响，在调用本机代码时提供较大的参数也可能会触发溢出。

5 多OS命令中使用的特殊元素处理不当（OS命令注入）

5.1 漏洞概述

漏洞编号	漏洞名称	风险等级
Header Manipulation: Cookies	多OS命令中使用的特殊元素处理不当（OS命令注入）	高危

5.2 漏洞详情

1）漏洞定位

BenchmarkTest00111.java:40

漏洞分析

漏洞爆发点:javax.servlet.http.Cookie.setPath()

爆发点函数:doGet()

缺陷源:

漏洞源代码

@WebServlet(value = "/sqli-00/BenchmarkTest00111") public class BenchmarkTest00111 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie userCookie = new javax.servlet.http.Cookie("BenchmarkTest00111", "bar"); userCookie.setMaxAge(60 * 3); // Store cookie for 3 minutes userCookie.setSecure(true); userCookie.setPath(request.getRequestURI()); userCookie.setDomain(new java.net.URL(request.getRequestURL().toString()).getHost()); response.addCookie(userCookie); javax.servlet.RequestDispatcher rd = request.getRequestDispatcher("/sqli-00/BenchmarkTest00111.html"); rd.include(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie[] theCookies = request.getCookies(); String param = "noCookieValueSupplied";

2）漏洞定位

BenchmarkTest00111.java:41

漏洞分析

漏洞爆发点:javax.servlet.http.Cookie.setDomain()

爆发点函数:doGet()

缺陷源:

漏洞源代码

@WebServlet(value = "/sqli-00/BenchmarkTest00111") public class BenchmarkTest00111 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie userCookie = new javax.servlet.http.Cookie("BenchmarkTest00111", "bar"); userCookie.setMaxAge(60 * 3); // Store cookie for 3 minutes userCookie.setSecure(true); userCookie.setPath(request.getRequestURI()); userCookie.setDomain(new java.net.URL(request.getRequestURL().toString()).getHost()); response.addCookie(userCookie); javax.servlet.RequestDispatcher rd = request.getRequestDispatcher("/sqli-00/BenchmarkTest00111.html"); rd.include(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie[] theCookies = request.getCookies(); String param = "noCookieValueSupplied"; if (theCookies != null) {

5.3 漏洞描述

命令执行/注入漏洞通常因为应用在服务器上拼接系统命令而造成的漏洞。攻击者通过提交恶意构造的参数破坏命令语句结构，从而达到执行恶意命令的目的。

5.4 修复建议

1. 在代码级调用shell时，对命令行中的特殊字符（比如|、&、；等）进行转义，防止执行其他非法命令。 2. 根据业务逻辑进行白名单方式校验或使用正则表达式进行过滤。 3. PHP中可使、用escapeshellarg、escapeshellcmd来转义对应敏感字符。 4. 对于相关敏感的命令执行函数，做好参数校验和合法性验证，或者直接在配置文件中禁用该函数，不要让用户可以直接控制eval、system、exec、shell_exec等函数的参数 。

6 对SQL命令中使用的特殊元素处理不当(SQL注入)

6.1 漏洞概述

漏洞编号	漏洞名称	风险等级
Missing Check against Null	对SQL命令中使用的特殊元素处理不当(SQL注入)	高危

6.2 漏洞详情

1）漏洞定位

BenchmarkTest00512.java:43

漏洞分析

漏洞爆发点:map = getParameterMap() : ServletRequest.getParameterMap may return NULL

爆发点函数:doPost()

缺陷源:

漏洞源代码

private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); java.util.Map<String, String[]> map = request.getParameterMap(); String param = ""; if (!map.isEmpty()) { String[] values = map.get("BenchmarkTest00512"); if (values != null) param = values[0]; } String bar = param; String sql = " SELECT * from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); statement.execute(sql); org.owasp.benchmark.helpers.DatabaseHelper.printResults(statement, sql, response);

2）漏洞定位

BenchmarkTest00510.java:43

漏洞分析

漏洞爆发点:map = getParameterMap() : ServletRequest.getParameterMap may return NULL

爆发点函数:doPost()

缺陷源:

漏洞源代码

private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); java.util.Map<String, String[]> map = request.getParameterMap(); String param = ""; if (!map.isEmpty()) { String[] values = map.get("BenchmarkTest00510"); if (values != null) param = values[0]; } String bar; String guess = "ABC"; char switchTarget = guess.charAt(2); // Simple case statement that assigns param to bar on conditions 'A', 'C', or 'D' switch (switchTarget) { case 'A': bar = param; break;

3）漏洞定位

BenchmarkTest01381.java:43

漏洞分析

漏洞爆发点:map = getParameterMap() : ServletRequest.getParameterMap may return NULL

爆发点函数:doPost()

缺陷源:

漏洞源代码

private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse

response) throws ServletException, IOException {

response.setContentType("text/html;charset=UTF-8"); java.util.Map<String, String[]> map = request.getParameterMap(); String param = ""; if (!map.isEmpty()) { String[] values = map.get("BenchmarkTest01381"); if (values != null) param = values[0]; } String bar = new Test().doSomething(request, param); String sql = "SELECT * from USERS where USERNAME=? and PASSWORD='" + bar + "'"; try { java.sql.Connection connection = org.owasp.benchmark.helpers.DatabaseHelper.getSqlConnection(); java.sql.PreparedStatement statement = connection.prepareStatement(sql, java.sql.Statement.RETURN_GENERATED_KEYS);

6.3 漏洞描述

SQL注入漏洞产生的原因是网站应用程序在编写时未对用户提交至服务器的数据进行合法性校验（类型、长度、业务参数合法性等），同时没有对用户输入数据进行有效地特殊字符过滤，使得用户的输入直接带入数据库执行，超出了SQL语句原来设计的预期结果，导致了SQL注入漏洞。

6.4 修复建议

1.输入验证 类型判断：字符型、整型； 长度判断：设置最大长度值； 业务参数合法性判断：比如支付金额不可能为负值这种； 特殊字符过滤：比如’，"，\，<，>，&，*，;，#，select，from，where，sub，if，union，sleep，and，or等； 验证所有的输入点，包括UA、Cookie以及其他HTTP头； 2.预编译处理&参数化查询 所有与数据库交互的业务接口均采用参数化查询，参数化的语句使用参数而不是将用户输入变量嵌入到SQL语句中，参数化查询是防御SQL 注入的最佳方法，比如：Java中的PreparedStatement，PHP中的PDO等。 3.使用最小权限原则 4.禁用敏感函数 5.进行权限控制 6.编码统一

7 释放后使用

7.1 漏洞概述

漏洞编号	漏洞名称	风险等级
Poor Style: Value Never Read	释放后使用	中危

7.2 漏洞详情

1）漏洞定位

BenchmarkTest01560.java:55

漏洞分析

漏洞爆发点:VariableAccess: count

爆发点函数:doPost()

缺陷源:

漏洞源代码

response.setContentType("text/html;charset=UTF-8");

org.owasp.benchmark.helpers.SeparateClassRequest scr = new

org.owasp.benchmark.helpers.SeparateClassRequest(request); String param =

scr.getTheParameter("BenchmarkTest01560"); if (param == null) param = "";

String bar = new Test().doSomething(request, param); String sql = "INSERT

INTO users (username, password) VALUES ('foo','" + bar + "')"; try {

java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.ge

tSqlStatement(); int count = statement.executeUpdate(sql,

java.sql.Statement.RETURN_GENERATED_KEYS);

org.owasp.benchmark.helpers.DatabaseHelper.outputUpdateComplete(sql,

response); } catch (java.sql.SQLException e) {
if(org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors)
{response.getWriter().println("Error processing request."); return; }
elsethrow new ServletException(e); } } // end doPost private class Test
{public String doSomething(HttpServletRequest request, String param)
throwsServletException, IOException { String bar;

2）漏洞定位

BenchmarkTest00441.java:55

漏洞分析

漏洞爆发点:VariableAccess: count

爆发点函数:doPost()

缺陷源:

漏洞源代码

response.setContentType("text/html;charset=UTF-8"); String param =
request.getParameter("BenchmarkTest00441"); if (param == null) param = "";
org.owasp.benchmark.helpers.ThingInterface thing =
org.owasp.benchmark.helpers.ThingFactory.createThing(); String bar =
thing.doSomething(param); String sql = "INSERT INTO users (username,
password) VALUES ('foo','" + bar + "')"; try { java.sql.Statement statement
= org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); int count =
statement.executeUpdate(sql, new int[] {1, 2});
org.owasp.benchmark.helpers.DatabaseHelper.outputUpdateComplete(sql,
response); } catch (java.sql.SQLException e) { if
(org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) {

response.getWriter().println("Error processing request."); return; } elsethrow new ServletException(e); } } }

3）漏洞定位

BenchmarkTest00111.java:82

漏洞分析

漏洞爆发点:VariableAccess: count

爆发点函数:doPost()

缺陷源:

漏洞源代码

valuesList.add("safe"); valuesList.add(param); valuesList.add("moresafe"); valuesList.remove(0); // remove the 1st safe value bar = valuesList.get(0); // get the param value } String sql = " INSERT INTO users (username, password) VALUES ('foo','" + bar + "')"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); int count = statement.executeUpdate(sql); org.owasp.benchmark.helpers.DatabaseHelper.outputUpdateComplete(sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing request."); return; } else throw new ServletException(e); } } }

4）漏洞定位

BenchmarkTest00203.java:63

漏洞分析

漏洞爆发点:VariableAccess: count

爆发点函数:doPost()

缺陷源:

漏洞源代码

```
param = java.net.URLDecoder.decode(param, "UTF-8"); String bar; // Simple ? condition that assigns param to bar on false condition int num = 106; bar = (7 * 42) - num > 200 ? "This should never happen" : param; String sql = "INSERT INTO users (username, password) VALUES ('foo','" + bar + "')"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); int count = statement.executeUpdate(sql, new String[] {"USERNAME", "PASSWORD"}); org.owasp.benchmark.helpers.DatabaseHelper.outputUpdateComplete(sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing request."); return; } else throw new ServletException(e); } } }
```

7.3 漏洞描述

在释放内存后引用内存可能会导致程序崩溃、使用意外值或执行代码。 使用先前释放的内存可能会产生许多不良后果，从有效数据的损坏到任意代码的执行，具体取决于缺陷的实例化和时间。可能发生数据损坏的最简单方法涉及系统重用释放的内存。无用后使用错误有两个常见且有时重叠的原因： 在这种情况下，有问题的内存在释放后的某个时间点有效地分配给另一个指针。指向释放的内存的原始指针将再次使用，并指向新分配中的某个位置。当数据被更改时，它会损坏有效使用的内存;这会在过程中诱发未定义的行为。 如果新分配的数据有机会容纳一个类，例如在C++中，各种函数指针可能分散在堆数据中。如果这些函数指针之一被有效外壳代码的地址覆盖，则可以实现任意代码的执行。

7.4 修复建议

1.选择提供自动内存管理的语言。 2.释放指针时，请确保在释放指针后将其设置为 NULL。但是，使用多个或复杂的数据结构可能会降低此策略的有用性。

8 对受限制目录的路径名的不当限制（“路径遍历”）

8.1 漏洞概述

漏洞编号	漏洞名称	风险等级
Resource Injection	对受限制目录的路径名的不当限制（“路径遍历”）	高危

8.2 漏洞详情

1）漏洞定位

BenchmarkTest00111.java:41

漏洞分析

漏洞爆发点:java.net.URL.URL()

爆发点函数:doGet()

缺陷源:

漏洞源代码

@WebServlet(value = "/sqli-00/BenchmarkTest00111") public class BenchmarkTest00111 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie userCookie = new javax.servlet.http.Cookie("BenchmarkTest00111", "bar");

```
userCookie.setMaxAge(60 * 3); // Store cookie for 3
minutesuserCookie.setSecure(true); userCookie.setPath(request.getRequestURI()
);userCookie.setDomain(new java.net.URL(request.getRequestURL().toString()).
getHost()); response.addCookie(userCookie);
javax.servlet.RequestDispatcherrd = request.getRequestDispatcher("/sqli-00/B
enchmarkTest00111.html");rd.include(request, response); } @Override public
voiddoPost(HttpServletRequest request, HttpServletResponse response)
throwsServletException, IOException { response.setContentType("text/html;cha
rset=UTF-8"); javax.servlet.http.Cookie[] theCookies =
request.getCookies();String param = "noCookieValueSupplied"; if (theCookies !
= null) {
```

8.3 漏洞描述

路径遍历是指应用程序接收了未经合理校验的用户参数用于进行与文件读取查看相关操作，而该参数包含了特殊的字符(例如“..”和“/”)，使用了这类特殊字符可以摆脱受保护的限制，越权访问一些受保护的文件、目录或者覆盖敏感数据。

8.4 修复建议

1.程序对非受信的用户输入数据净化，对网站用户提交过来的文件名进行硬编码或者统一编码，过滤非法字符。 2.对文件后缀进行白名单控制，对包含了恶意的符号或者空字节进行拒绝。 3.合理配置 web 服务器的目录权限。

9 跨站点请求伪造（CSRF）

9.1 漏洞概述

漏洞编号	漏洞名称	风险等级
SQL Injection	跨站点请求伪造（CSRF）	中危

9.2 漏洞详情

1）漏洞定位

BenchmarkTest00111.java:82

漏洞分析

漏洞爆发点:java.sql.Statement.executeUpdate()

爆发点函数:doPost()

缺陷源:

漏洞源代码

valuesList.add("safe"); valuesList.add(param); valuesList.add("moresafe"); valuesList.remove(0); // remove the 1st safe value bar = valuesList.get(0); // get the param value } String sql = " INSERT INTO users (username, password) VALUES ('foo','" + bar + "')"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); int count = statement.executeUpdate(sql); org.owasp.benchmark.helpers.DatabaseHelper.outputUpdateComplete(sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing request."); return; } else throw new ServletException(e); } } }

2）漏洞定位

BenchmarkTest00512.java:57

漏洞分析

漏洞爆发点:java.sql.Statement.execute()

爆发点函数:doPost()

缺陷源:

漏洞源代码

```
java.util.Map<String, String[]> map = request.getParameterMap(); String param = ""; if (!map.isEmpty()) { String[] values = map.get("BenchmarkTest00512"); if (values != null) param = values[0]; } String bar = param; String sql = "SELECT * from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); statement.execute(sql); org.owasp.benchmark.helpers.DatabaseHelper.printResults(statement, sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing request."); return; } else throw new ServletException(e); } } }
```

3）漏洞定位

BenchmarkTest01970.java:58

漏洞分析

漏洞爆发点:java.sql.Statement.addBatch()

爆发点函数:doPost()

缺陷源:

漏洞源代码

```
if (request.getHeader("BenchmarkTest01970") != null) { param = request.getHeader("BenchmarkTest01970"); } // URL Decode the header value since req.getHeader() doesn't. Unlike req.getParameter(). param = java.net.URLDecoder.decode(param, "UTF-8"); String bar =
```

doSomething(request, param); String sql = "SELECT * from USERS whereUSERNAME='foo' and PASSWORD='" + bar + "'"; try { java.sql.Statementstatement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement();statement.addBatch(sql); int[] counts = statement.executeBatch();org.owasp.benchmark.helpers.DatabaseHelper.printResults(sql, counts,response); } catch (java.sql.SQLException e) { if(org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) {response.getWriter().println("Error processing request."); return; } elsethrow new ServletException(e); } } // end doPost private static StringdoSomething(HttpServletRequest request, String param) throwsServletException, IOException { String bar; String guess = "ABC";

4）漏洞定位

BenchmarkTest00515.java:67

漏洞分析

漏洞爆发点:java.sql.Statement.execute()

爆发点函数:doPost()

缺陷源:

漏洞源代码

valuesList.add("safe"); valuesList.add(param); valuesList.add("moresafe"); valuesList.remove(0); // remove the 1st safe value bar = valuesList.get(0); // get the param value } String sql = " SELECT * from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); statement.execute(sql, new int[] {1, 2});

org.owasp.benchmark.helpers.DatabaseHelper.printResults(statement, sql,response); } catch (java.sql.SQLException e) { if(org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) {response.getWriter().println("Error processing request."); return; } elsethrow new ServletException(e); } } }

5）漏洞定位

BenchmarkTest00850.java:87

漏洞分析

漏洞爆发点:java.sql.Statement.executeQuery()

爆发点函数:doPost()

缺陷源:

漏洞源代码

valuesList.add("safe"); valuesList.add(param); valuesList.add("moresafe"); valuesList.remove(0); // remove the 1st safe value bar = valuesList.get(0); // get the param value } String sql = " SELECT * from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); java.sql.ResultSet rs = statement.executeQuery(sql); org.owasp.benchmark.helpers.DatabaseHelper.printResults(rs, sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing request."); return; } else throw new ServletException(e); } } }

6）漏洞定位

BenchmarkTest00510.java:78

漏洞分析

漏洞爆发点:java.sql.Connection.prepareCall()

爆发点函数:doPost()

缺陷源:

漏洞源代码

bar = param; break; default: bar = "bobs_your_uncle"; break; } String sql = "{call " + bar + "}"; try { java.sql.Connection connection = org.owasp.benchmark.helpers.DatabaseHelper.getSqlConnection(); java.sql.CallableStatement statement = connection.prepareCall(sql, java.sql.ResultSet.TYPE_FORWARD_ONLY, java.sql.ResultSet.CONCUR_READ_ONLY, java.sql.ResultSet.CLOSE_CURSORS_AT_COMMIT); java.sql.ResultSet rs = statement.executeQuery(); org.owasp.benchmark.helpers.DatabaseHelper.printResults(rs, sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing request."); return; } else throw new ServletException(e); } } }

7）漏洞定位

BenchmarkTest01091.java:58

漏洞分析

漏洞爆发点:java.sql.Statement.execute()

爆发点函数:doPost()

缺陷源:

漏洞源代码

if (request.getHeader("BenchmarkTest01091") != null) { param = request.getHeader("BenchmarkTest01091"); } // URL Decode the header value since req.getHeader() doesn't. Unlike req.getParameter(). param = java.net.URLDecoder.decode(param, "UTF-8"); String bar = new Test().doSomething(request, param); String sql = "SELECT * from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); statement.execute(sql); org.owasp.benchmark.helpers.DatabaseHelper.printResults(statement, sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing request."); return; } else throw new ServletException(e); } } // end doPost private class Test { public String doSomething(HttpServletRequest request, String param) throws ServletException, IOException { String bar = "";

8）漏洞定位

BenchmarkTest01381.java:58

漏洞分析

漏洞爆发点:java.sql.Connection.prepareStatement()

爆发点函数:doPost()

缺陷源:

漏洞源代码

String param = ""; if (!map.isEmpty()) { String[] values = map.get("BenchmarkTest01381"); if (values != null) param = values[0]; } String bar = new Test().doSomething(request, param); String sql = "SELECT *

from USERS where USERNAME=? and PASSWORD='" + bar + "'"; try {java.sql.Connection connection = org.owasp.benchmark.helpers.DatabaseHelper.getSqlConnection(); java.sql.PreparedStatement statement =connection.prepareStatement(sql, java.sql.Statement.RETURN_GENERATED_KEYS); statement.setString(1, "foo"); statement.execute();org.owasp.benchmark.helpers.DatabaseHelper.printResults(statement, sql,response); } catch (java.sql.SQLException e) { if(org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) {response.getWriter().println("Error processing request."); return; } elsethrow new ServletException(e); } } // end doPost private class Test {public String doSomething(HttpServletRequest request, String param) throwsServletException, IOException {

9）漏洞定位

BenchmarkTest00203.java:63

漏洞分析

漏洞爆发点:java.sql.Statement.executeUpdate()

爆发点函数:doPost()

缺陷源:

漏洞源代码

param = java.net.URLDecoder.decode(param, "UTF-8"); String bar; // Simple ? condition that assigns param to bar on false condition int num = 106; bar = (7 * 42) - num > 200 ? "This should never happen" : param; String sql = "INSERT INTO users (username, password) VALUES ('foo','" + bar + "')"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); int count =

statement.executeUpdate(sql, new String[] {"USERNAME", "PASSWORD"}); org.owasp.benchmark.helpers.DatabaseHelper.outputUpdateComplete(sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing request."); return; } else throw new ServletException(e); } } }

10）漏洞定位

BenchmarkTest00008.java:57

漏洞分析

漏洞爆发点:java.sql.Connection.prepareCall()

爆发点函数:doPost()

缺陷源:

漏洞源代码

String param = ""; if (request.getHeader("BenchmarkTest00008") != null) { param = request.getHeader("BenchmarkTest00008"); } // URL Decode the header value since req.getHeader() doesn't. Unlike req.getParameter(). param = java.net.URLDecoder.decode(param, "UTF-8"); String sql = "{call " + param + "}"; try { java.sql.Connection connection = org.owasp.benchmark.helpers.DatabaseHelper.getSqlConnection(); java.sql.CallableStatement statement = connection.prepareCall(sql); java.sql.ResultSet rs = statement.executeQuery(); org.owasp.benchmark.helpers.DatabaseHelper.printResults(rs, sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing

request."); return; } else throw new ServletException(e); } } }

11）漏洞定位

BenchmarkTest00441.java:55

漏洞分析

漏洞爆发点:executeUpdate()

爆发点函数:doPost()

缺陷源:

漏洞源代码

response.setContentType("text/html;charset=UTF-8"); String param = request.getParameter("BenchmarkTest00441"); if (param == null) param = ""; org.owasp.benchmark.helpers.ThingInterface thing = org.owasp.benchmark.helpers.ThingFactory.createThing(); String bar = thing.doSomething(param); String sql = "INSERT INTO users (username, password) VALUES ('foo','" + bar + "')"; try { java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement(); int count = statement.executeUpdate(sql, new int[] {1, 2}); org.owasp.benchmark.helpers.DatabaseHelper.outputUpdateComplete(sql, response); } catch (java.sql.SQLException e) { if (org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) { response.getWriter().println("Error processing request."); return; } else throw new ServletException(e); } } }

12）漏洞定位

BenchmarkTest01560.java:55

漏洞分析

漏洞爆发点:executeUpdate()

爆发点函数:doPost()

缺陷源:

漏洞源代码

```
response.setContentType("text/html;charset=UTF-8");
org.owasp.benchmark.helpers.SeparateClassRequest scr = new
org.owasp.benchmark.helpers.SeparateClassRequest(request); String param =
scr.getTheParameter("BenchmarkTest01560"); if (param == null) param = "";
String bar = new Test().doSomething(request, param); String sql = "INSERT
INTO users (username, password) VALUES ('foo','" + bar + "')"; try {
java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.ge
tSqlStatement(); int count = statement.executeUpdate(sql,
java.sql.Statement.RETURN_GENERATED_KEYS);
org.owasp.benchmark.helpers.DatabaseHelper.outputUpdateComplete(sql,
response); } catch (java.sql.SQLException e) { if
(org.owasp.benchmark.helpers.DatabaseHelper.hideSQLErrors) {
response.getWriter().println("Error processing request."); return; } else
throw new ServletException(e); } } // end doPost private class Test {
public String doSomething(HttpServletRequest request, String param) throws
ServletException, IOException { String bar;
```

9.3 漏洞描述

CSRF（Cross-site request forgery）跨站请求伪造，也被称为“One Click Attack”或者Session Riding，通常缩写为CSRF，是一种对网站的恶意利用。尽管听起来像跨站脚本（XSS），但它与XSS非常不同，XSS利用站点内的信任用户，而CSRF则通过伪装来自受信任用户的请求来自受信任的网站。攻击者通过伪造来自受信任用户的请

求，达到增加、删除、篡改网站内容的目的。

9.4 修复建议

1.验证HTTP Referer字段； 2. 关键业务操作使用图形验证码进行二次确认验证，只接受 POST 请求，GET请求应只浏览而不改变服务器端资源； 3. 过滤用户输入，不允许发布含有站内操作URL的链接； 4. 增加csrftoken验证，csrftoken可以在用户登陆后产生并放于 session 之中，然后在每次请求时把csrftoken从 session 中拿出， 与请求中的csrftoken进行对比。对于GET请求，csrftoken将附在请求地址之后，对于POST请求来说，要在form的最后加上 ； 5. 在 HTTP 头中自定义属性并验证，该用法同csrftoken，同为随机值校验，但是这里并不是把csrftoken以参数的形式置于HTTP请求 之中，而是把它放到 HTTP头中自定义的属性里； 6. 在浏览其它站点前登出站点或者在浏览器会话结束后清理浏览器的cookie。

10 危险类型文件的不加限制上传

10.1 漏洞概述

漏洞编号	漏洞名称	风险等级
rmation Leak: Incomplete Servlet Error Handling	危险类型文件的不加限制上传	高危

10.2 漏洞详情

1）漏洞定位

BenchmarkTest00026.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-00/BenchmarkTest00026") public class BenchmarkTest00026 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { // some code response.setContentType("text/html;charset=UTF-8"); String param = request.getParameter("BenchmarkTest00026"); if (param == null) param = ""; String sql = "SELECT * from USERS where USERNAME='foo' and PASSWORD='" + param + "'"; try { org.springframework.jdbc.support.rowset.SqlRowSet results = org.owasp.benchmark.helpers.DatabaseHelper.JDBCtemplate.queryForRowSet(sql); response.getWriter().println("Your results are: "); // System.out.println("Your results are"); while (results.next()) {

2）漏洞定位

BenchmarkTest01560.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-03/BenchmarkTest01560") public class BenchmarkTest01560 extends

HttpServlet { private static final long serialVersionUID = 1L; @Overridepublic void doGet(HttpServletRequest request, HttpServletResponse response)throws ServletException, IOException { doPost(request, response); }@Override public void doPost(HttpServletRequest request,HttpServletResponse response) throws ServletException, IOException {response.setContentType("text/html;charset=UTF-8");org.owasp.benchmark.helpers.SeparateClassRequest scr = neworg.owasp.benchmark.helpers.SeparateClassRequest(request); String param =scr.getTheParameter("BenchmarkTest01560"); if (param == null) param = "";String bar = new Test().doSomething(request, param); String sql = "INSERTINTO users (username, password) VALUES ('foo','" + bar + "')"; try {java.sql.Statement statement = org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement();

3）漏洞定位

BenchmarkTest00441.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-00/BenchmarkTest00441") public class BenchmarkTest00441 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); }

@Override public void doPost(HttpServletRequest request,HttpServletResponse response) throws ServletException, IOException {response.setContentType("text/html;charset=UTF-8"); String param =request.getParameter("BenchmarkTest00441"); if (param == null) param = "";org.owasp.benchmark.helpers.ThingInterface thing =org.owasp.benchmark.helpers.ThingFactory.createThing(); String bar =thing.doSomething(param); String sql = "INSERT INTO users (username,password) VALUES ('foo','" + bar + "')"; try { java.sql.Statement statement= org.owasp.benchmark.helpers.DatabaseHelper.getSqlStatement();

4）漏洞定位

BenchmarkTest00510.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-00/BenchmarkTest00510") public class BenchmarkTest00510 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); java.util.Map<String,

String[]> map = request.getParameterMap(); String param = ""; if (!map.isEmpty()) { String[] values = map.get("BenchmarkTest00510"); if (values != null) param = values[0]; } String bar; String guess = "ABC"; char switchTarget = guess.charAt(2); // Simple case statement that assigns param to bar on conditions 'A', 'C', or 'D'

5）漏洞定位

BenchmarkTest00111.java:33

漏洞分析

漏洞爆发点:Function: doGet

爆发点函数:doGet()

缺陷源:

漏洞源代码

import java.io.IOException; import javax.servlet.ServletException; import javax.servlet.annotation.WebServlet; import javax.servlet.http.HttpServlet; import javax.servlet.http.HttpServletRequest; import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-00/BenchmarkTest00111") public class BenchmarkTest00111 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie userCookie = new javax.servlet.http.Cookie("BenchmarkTest00111", "bar"); userCookie.setMaxAge(60 * 3); // Store cookie for 3 minutes userCookie.setSecure(true); userCookie.setPath(request.getRequestURI());

userCookie.setDomain(new java.net.URL(request.getRequestURL().toString()).getHost()); response.addCookie(userCookie); javax.servlet.RequestDispatcherrd = request.getRequestDispatcher("/sqli-00/BenchmarkTest00111.html");rd.include(request, response); } @Override

6）漏洞定位

BenchmarkTest01970.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-04/BenchmarkTest01970") public class BenchmarkTest01970 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); String param = ""; if (request.getHeader("BenchmarkTest01970") != null) { param = request.getHeader("BenchmarkTest01970"); } // URL Decode the header value since req.getHeader() doesn't. Unlike req.getParameter(). param = java.net.URLDecoder.decode(param, "UTF-8"); String bar = doSomething(request, param); String sql = "SELECT * from USERS where

USERNAME='foo' and PASSWORD='" + bar + "'";

7）漏洞定位

BenchmarkTest01381.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-02/BenchmarkTest01381") public class BenchmarkTest01381 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); java.util.Map<String, String[]> map = request.getParameterMap(); String param = ""; if (!map.isEmpty()) { String[] values = map.get("BenchmarkTest01381"); if (values != null) param = values[0]; } String bar = new Test().doSomething(request, param); String sql = "SELECT * from USERS where USERNAME=? and PASSWORD='" + bar + "'"; try {

8）漏洞定位

BenchmarkTest00512.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-01/BenchmarkTest00512") public class BenchmarkTest00512 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); java.util.Map<String, String[]> map = request.getParameterMap(); String param = ""; if (!map.isEmpty()) { String[] values = map.get("BenchmarkTest00512"); if (values != null) param = values[0]; } String bar = param; String sql = " SELECT * from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; try {

9）漏洞定位

BenchmarkTest01091.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-02/BenchmarkTest01091") public class BenchmarkTest01091 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); String param = ""; if (request.getHeader("BenchmarkTest01091") != null) { param = request.getHeader("BenchmarkTest01091"); } // URL Decode the header value since req.getHeader() doesn't. Unlike req.getParameter(). param = java.net.URLDecoder.decode(param, "UTF-8"); String bar = new Test().doSomething(request, param); String sql = "SELECT * from USERS where USERNAME='foo' and PASSWORD='" + bar + "'";

10）漏洞定位

BenchmarkTest00767.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-01/BenchmarkTest00767") public class BenchmarkTest00767 extends HttpServlet { private static final long serialVersionUID = 1L; @Override

public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); String[] values = request.getParameterValues("BenchmarkTest00767"); String param; if (values ! = null && values.length > 0) param = values[0]; else param = ""; String bar; String guess = "ABC"; char switchTarget = guess.charAt(2); // Simple case statement that assigns param to bar on conditions 'A', 'C', or 'D' switch (switchTarget) { case 'A':

11）漏洞定位

BenchmarkTest00111.java:49

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie userCookie = new javax.servlet.http.Cookie("BenchmarkTest00111", "bar"); userCookie.setMaxAge(60 * 3); // Store cookie for 3 minutes userCookie.setSecure(true); userCookie.setPath(request.getRequestURI()); userCookie.setDomain(new java.net.URL(request.getRequestURL().toString()).getHost()); response.addCookie(userCookie); javax.servlet.RequestDispatcher

rd = request.getRequestDispatcher("/sqli-00/BenchmarkTest00111.html");rd.include(request, response); } @Override public voiddoPost(HttpServletRequest request, HttpServletResponse response) throwsServletException, IOException { response.setContentType("text/html;charset=UTF-8"); javax.servlet.http.Cookie[] theCookies = request.getCookies();String param = "noCookieValueSupplied"; if (theCookies != null) { for(javax.servlet.http.Cookie theCookie : theCookies) { if(theCookie.getName().equals("BenchmarkTest00111")) { param =java.net.URLDecoder.decode(theCookie.getValue(), "UTF-8"); break; } } }

12）漏洞定位

BenchmarkTest00850.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-01/BenchmarkTest00850") public class BenchmarkTest00850 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); String queryString =

request.getQueryString(); String paramval = "BenchmarkTest00850" + "=";
intparamLoc = -1; if (queryString != null) paramLoc
=queryString.indexOf(paramval); if (paramLoc == -1) { response.getWriter() .
println("getQueryString() couldn't find expected parameter '" + "
BenchmarkTest00850" + "' in query string."); return; }

13）漏洞定位

BenchmarkTest00203.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "
/sqli-00/BenchmarkTest00203") public class BenchmarkTest00203 extends
HttpServlet { private static final long serialVersionUID = 1L; @Override
public void doGet(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException { doPost(request, response); }
@Override public void doPost(HttpServletRequest request,
HttpServletResponse response) throws ServletException, IOException {
response.setContentType("text/html;charset=UTF-8"); String param = ""; if
(request.getHeader("BenchmarkTest00203") != null) { param =
request.getHeader("BenchmarkTest00203"); } // URL Decode the header value
since req.getHeader() doesn't. Unlike req.getParameter(). param =
java.net.URLDecoder.decode(param, "UTF-8"); String bar; // Simple ?

condition that assigns param to bar on false condition int num = 106;

14）漏洞定位

BenchmarkTest00840.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-01/BenchmarkTest00840") public class BenchmarkTest00840 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); String queryString = request.getQueryString(); String paramval = "BenchmarkTest00840" + "="; int paramLoc = -1; if (queryString != null) paramLoc = queryString.indexOf(paramval); if (paramLoc == -1) { response.getWriter() . println("getQueryString() couldn't find expected parameter '" + " BenchmarkTest00840" + "' in query string."); return; }

15）漏洞定位

BenchmarkTest00008.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-00/BenchmarkTest00008") public class BenchmarkTest00008 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { // some code response.setContentType("text/html;charset=UTF-8"); String param = ""; if (request.getHeader("BenchmarkTest00008") != null) { param = request.getHeader("BenchmarkTest00008"); } // URL Decode the header value since req.getHeader() doesn't. Unlike req.getParameter(). param = java.net.URLDecoder.decode(param, "UTF-8"); String sql = "{call " + param + "}"; try {

16）漏洞定位

BenchmarkTest00764.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-01/BenchmarkTest00764") public class BenchmarkTest00764 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); String[] values = request.getParameterValues("BenchmarkTest00764"); String param; if (values != null && values.length > 0) param = values[0]; else param = ""; String bar = param; String sql = "SELECT userid from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; try { // Long results = // org.owasp.benchmark.helpers.DatabaseHelper.JDBCtemplate.queryForLong(sql); Long results =

17）漏洞定位

BenchmarkTest00515.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-01/BenchmarkTest00515") public class BenchmarkTest00515 extends HttpServlet { private static final long serialVersionUID = 1L; @Override

public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); java.util.Map<String, String[]> map = request.getParameterMap(); String param = ""; if (!map.isEmpty()) { String[] values = map.get("BenchmarkTest00515"); if (values != null) param = values[0]; } String bar = ""; if (param != null) { java.util.List<String> valuesList = new java.util.ArrayList<String>(); valuesList.add("safe"); valuesList.add(param);

18）漏洞定位

BenchmarkTest00765.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-01/BenchmarkTest00765") public class BenchmarkTest00765 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {

response.setContentType("text/html;charset=UTF-8"); String[] values = request.getParameterValues("BenchmarkTest00765"); String param; if (values ! = null && values.length > 0) param = values[0]; else param = ""; org.owasp.benchmark.helpers.ThingInterface thing = org.owasp.benchmark.helpers.ThingFactory.createThing(); String bar = thing.doSomething(param); String sql = "SELECT userid from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; try { // Long results =

19）漏洞定位

BenchmarkTest00768.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-01/BenchmarkTest00768") public class BenchmarkTest00768 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); String[] values = request.getParameterValues("BenchmarkTest00768"); String param; if (values ! = null && values.length > 0) param = values[0]; else param = "";

org.owasp.benchmark.helpers.ThingInterface thing = org.owasp.benchmark.helpers.ThingFactory.createThing(); String bar = thing.doSomething(param); String sql = "SELECT TOP 1 userid from USERS where USERNAME='foo' and PASSWORD='" + bar + "'"; try {

20）漏洞定位

BenchmarkTest00434.java:39

漏洞分析

漏洞爆发点:Function: doPost

爆发点函数:doPost()

缺陷源:

漏洞源代码

import javax.servlet.http.HttpServletResponse; @WebServlet(value = "/sqli-00/BenchmarkTest00434") public class BenchmarkTest00434 extends HttpServlet { private static final long serialVersionUID = 1L; @Override public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { doPost(request, response); } @Override public void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException { response.setContentType("text/html;charset=UTF-8"); String param = request.getParameter("BenchmarkTest00434"); if (param == null) param = ""; org.owasp.benchmark.helpers.ThingInterface thing = org.owasp.benchmark.helpers.ThingFactory.createThing(); String bar = thing.doSomething(param); try { String sql = "SELECT * from USERS where USERNAME='foo' and PASSWORD='" + bar + "'";

org.owasp.benchmark.helpers.DatabaseHelper.JDBCtemplate.batchUpdate(sql);response.getWriter()

10.3 漏洞描述

软件允许攻击者上传或传输可在产品环境中自动处理的危险类型的文件。

10.4 修复建议

1.为上传的文件生成新的唯一文件名，而不是使用用户提供的文件名，以便从根本上不使用外部输入。 2.当可接受的对象集（如文件名或 URL）受到限制或已知时，请创建从一组固定输入值（如数字 ID）到实际文件名或 URL 的映射，并拒绝所有其他输入。 3.考虑将上载的文件完全存储在 Web 文档根目录之外。然后，使用其他机制动态传递文件。 4.定义一组非常有限的允许扩展名，并且仅生成以这些扩展名结尾的文件名。 5.确保文件名中仅使用一个扩展名。一些Web服务器，包括某些版本的Apache，可能会根据内部扩展名处理文件，以便将“文件名.php.gif”提供给PHP解释器。